

CONTENTS

- Overview
- 1 The 2023 case, and why it was convincing
- 2 What fine-tuning does to a model
- 3 The 2025 rematch
- 4 Why general models caught up
- 5 Customisation without touching the weights
- 6 Agent skills: packaging procedural knowledge
- 7 LoRA and parameter-efficient fine-tuning
- 8 Where fine-tuning still wins
- 9 The decision order
- Glossary
- Check yourself
- Where to go next

Does Fine-Tuning Still Earn Its Place?

Work out where custom weights still beat a frontier model plus a good context stack, and where they no longer do.

For engineers and leads deciding whether to invest in custom model training · 26 July 2026

[Watch the source video](#)

[Read the condensed version](#)

[Read the highlights](#)

BEFORE YOU START

- What a large language model is and roughly how it is trained
- Familiarity with prompting and, ideally, RAG
- Comfort reading Python

Overview

In 2023 a well-known legal AI company fine-tuned a custom model in partnership with a frontier lab, and in blind tests attorneys preferred it over GPT-4 — the leading model at the time — 97% of the time. That looked like a settled argument. If a general model is not quite right for your domain, fine-tune it.

In 2025 the same company built a legal benchmark and tested that custom model against the latest frontier models. Seven general-purpose models, none of which had received any legal fine-tuning, had surpassed it. Bloomberg saw something comparable: they trained BloombergGPT from scratch, and later evaluations found GPT-4 and ChatGPT outperforming it on many financial benchmarks.

That is not an argument that fine-tuning is dead. It is an argument that the baseline moves, and that a custom model is a fixed artefact competing against a target that improves every few months. What changed is the size of the alternative: there is now a whole stack of customisation techniques that never touch a single weight, and most teams should exhaust it before considering training. Fine-tuning survives for a narrower and more specific set of reasons than it had in 2023.

KEY TAKEAWAYS

- ✓ Fine-tuning continues a base model's training on a focused dataset, adjusting its weights toward a narrow domain.
- ✓ A fine-tuned model is a fixed artefact competing against frontier models that keep improving — a moving target.
- ✓ Large context windows removed the main reason to bake documents into weights: you can now just supply them.
- ✓ Reasoning models shift capability to inference time, so 'how hard it thinks now' rivals 'how it was trained months ago'.
- ✓ RAG, context engineering and agent skills customise behaviour without modifying any weights.
- ✓ Agent skills package procedural knowledge as loadable files, so any general model can perform a specialised procedure.
- ✓ Most production fine-tuning today is LoRA or a related parameter-efficient method that trains a small adapter over frozen weights.
- ✓ Fine-tuning still wins for hard latency constraints, distillation into smaller models, and reinforcement fine-tuning against a programmatic grader.

01 The 2023 case, and why it was convincing

0:00

A custom legal model beat the leading frontier model in blind tests, decisively. The conclusion drawn was that domain work means fine-tuning.

The case study is worth taking seriously rather than dismissing with hindsight. In 2023 a well-known legal AI company worked with a frontier lab to fine-tune its own custom model on legal work. In blind tests, attorneys preferred the fine-tuned model over GPT-4 — the leading general model at the time — 97% of the time.

That is not a marginal result. It is a decisive one, produced by domain experts who did not know which system they were evaluating. The lesson everyone drew was reasonable: if a general-purpose model is not quite right for a specific use case, fine-tune it.

What makes the story instructive is that the result was real and the conclusion still expired. The comparison was against one fixed opponent at one moment. The custom model was a finished artefact; the general model was a snapshot of something under continuous development by an organisation with far more resources than any single customer. Both facts were visible in 2023 and neither seemed important, because nobody had yet watched a two-year-old fine-tune get overtaken.

KEY POINTS

- A 2023 fine-tuned legal model beat GPT-4 in blind expert testing, 97% of the time.
- The evaluation was sound: domain experts, blind, decisive margin.
- The conclusion — fine-tune when a general model is not quite right — followed reasonably.
- The comparison was against one fixed opponent at one moment in time.
- A custom model is finished; the general model it was measured against is not.

What the 2023 comparison actually established

A true statement with an implicit time bound that nobody wrote down. Reading it with the date attached makes the later result unsurprising rather than shocking.

custom legal model (2023) vs GPT-4 (2023)

→ attorneys preferred the custom model 97% of the time

established: a fine-tune beat that specific model at that specific time

assumed: the comparison would keep holding

unstated: only one side of it was still being developed

02 What fine-tuning does to a model

0:41

Take a base model trained on a broad corpus and continue training it on a focused dataset, producing a new model with adjusted weights.

Start with a base LLM — what comes off the shelf, whether from a frontier lab directly or as an open-source model. That base model was trained on an enormous corpus, effectively a scrape of the internet, so a great deal of general knowledge is baked into its weights.

Fine-tuning takes that base model and customises it by continuing its training, now on a much more focused dataset: legal contracts, an organisation's internal support tickets, the kind of material that is not sitting on the open web waiting to be scraped.

What comes out is a new model. It incorporates the information from the focused dataset plus all the base model's weights, now adjusted by that additional training. The result should be better at certain narrow tasks. That is the promise of fine-tuning, and within its terms it works.

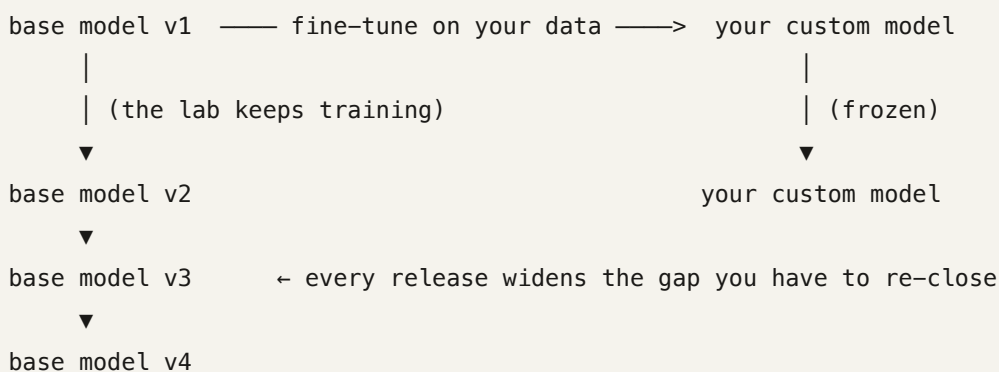
Two properties of the output matter for everything that follows. It is a distinct artefact — a separate model you now own, host, evaluate and maintain, forked from a base that will keep moving without you. And the customisation is baked in: the domain knowledge is distributed across the weights rather than sitting in a file, so updating it means training again, and inspecting it is not really possible.

KEY POINTS

- Start from a pre-trained base model with broad knowledge in its weights.
- Continue training on a focused dataset of domain material not present on the open web.
- The output is a new model: base weights adjusted by the additional training.
- It is a separate artefact to host, evaluate and maintain, forked from a moving base.
- The customisation is baked into weights — not inspectable, not incrementally updatable.

The fork

The picture worth carrying. At the moment of the fork the two are equivalent plus your domain data. From then on only one of them receives further investment.



What you now own

The part that rarely appears in the initial estimate. The training run is a one-off; everything below it is permanent.

- a model artefact to store and version
- hosting for it, separate from any provider endpoint
- an evaluation suite, because public benchmarks no longer describe your model
- a retraining pipeline, since every update is another run
- a regression process, because new data can degrade old behaviour
- a decision, every time the base model updates, about whether to re-fork

03 The 2025 rematch

2:10

The same company built a legal benchmark and found seven general-purpose models,

with no legal fine-tuning at all, had surpassed their custom one.

In 2025 the legal AI company created its own legal benchmark to measure how effective its models were at specific tasks, and tested the fine-tuned system against the latest crop of frontier models.

Seven of the general-purpose models had surpassed the company's custom model. Not models that had received better legal fine-tuning — models that had received none at all.

Bloomberg saw something similar. They famously trained BloombergGPT from scratch on financial data, and later evaluations found GPT-4 and ChatGPT outperforming it on many financial benchmarks. Two independent cases, two domains, the same shape of result.

The useful reading is not that fine-tuning does not work. It is that domain advantage decays relative to a baseline that keeps rising. A fine-tune buys you a lead over the base model as it stands today; general capability improvements accrue to everyone, including your competitors who did nothing. Whether your lead survives depends on how fast the frontier moves against how fast you can re-fork — and one of those you do not control.

One detail deserves credit: the company built its own benchmark. That is the right instinct regardless of which way the result falls. Public benchmarks cannot tell you whether a custom model is still worth its cost on your tasks, and without a domain-specific evaluation you will not find out that your lead has gone.

KEY POINTS

- Seven general models with no legal fine-tuning surpassed the custom model on the company's own benchmark.
- Bloomberg's from-scratch financial model was similarly overtaken on financial benchmarks.
- Fine-tuning still worked; the baseline it was measured against rose faster.
- General capability gains accrue to everyone, including those who invested nothing.
- A domain benchmark you own is the only way to detect that a custom model's lead has gone.

Two independent cases, one shape

Different domains, different organisations, different approaches — one fine-tuned, one trained from scratch. The pattern is about the baseline, not about any particular training decision.

legal AI company	Bloomberg
2023: fine-tuned model beats GPT-4 in 97% of blind expert comparisons	trained BloombergGPT from scratch on financial data
2025: 7 general models, none legally fine-tuned, beat it on the company's own benchmark	later evaluations found GPT-4 and ChatGPT outperforming it on many financial benchmarks

The evaluation that should run continuously

The check that turns this from a story into a control. Run it on every frontier release; the day the gap goes negative, the custom model is a cost with no benefit and should be retired rather than defended.

```
def custom_model_still_worth_it(benchmark):
    ours = benchmark.score(custom_model)
    best = max(benchmark.score(m) for m in current_frontier_models())

    print(f"custom {ours:.3f}    best general {best:.3f}    gap {ours-best:+.3f}")
    return ours > best + MIN_MARGIN          # margin must cover hosting,
                                           # retraining and maintenance

# run on every frontier release, not annually.
# a negative gap is a decommission decision, not a tuning problem.
```

04 Why general models caught up

3:35

Three changes: context windows grew enormously, reasoning moved to inference time, and inference got cheaper.

The first change is context. The original GPT-3 had a window of 2,000 tokens. Frontier models now routinely handle a million and more. If a model can read 500 pages of legal documents directly in its prompt, why bake those documents into the weights at all? Just pass whatever is contextually relevant at

prompt time. A great deal of what fine-tuning was used for in 2023 was knowledge injection, and knowledge injection is what a large context window does natively, without a training run and without going stale.

The second is reasoning models, which do extended thinking at inference time — working through a problem step by step before answering. That relocates where capability comes from. Performance now depends substantially on how hard the model thinks at the moment of the question, rather than only on how it was trained months earlier. Extra thinking is available to anyone, on any prompt, with no data collection.

The third is simply that inference keeps getting cheaper and models keep getting more efficient. Combine the three and training a custom version becomes a moving target: by the time the fine-tuned model ships, the next frontier release may already have leapfrogged it.

That last point is the one that changes planning rather than architecture. A fine-tuning project has a lead time — data collection, training, evaluation, deployment — and the baseline improves during it. If your cycle is six months and meaningful releases arrive more often than that, you are not building a lead, you are running to stay level.

KEY POINTS

- Context windows went from around 2,000 tokens to a million and beyond.
- Knowledge that used to be baked into weights can simply be supplied in the prompt.
- Reasoning models move capability to inference time, available on any prompt without training.
- Inference keeps getting cheaper and models more efficient.
- A fine-tuning project's lead time competes with the frontier's release cadence.

What a 2,000-token window forced

The constraint that produced the 2023 consensus. When you cannot fit the material, weights are the only place to put it — which was true then and stopped being true without the advice being updated.

GPT-3 (2020)	2,000 tokens	≈ 3 pages
frontier (now)	1,000,000+	≈ 1,500 pages

in 2020: the domain corpus cannot fit → put it in the weights

now: 500 pages of contracts fit in the prompt, fresh every time,

with no training run and nothing to go stale

The moving-target arithmetic

Run this with your own numbers before committing to a fine-tuning programme. If the lead time exceeds the release cadence, the model ships into a world where the baseline has already moved.

data collection and cleaning	6–12 weeks	(mostly human effort)
training and hyperparameter runs	1–3 weeks	
evaluation and regression checks	2–4 weeks	
deployment and rollout	1–2 weeks	
		10–21 weeks

meaningful frontier releases arrive more often than that.

you are not building a lead – you are re-closing a gap.

05 Customisation without touching the weights

5:41

RAG supplies the knowledge, and context engineering assembles everything the model needs into a deliberately constructed prompt.

If we are not adjusting weights, how do we make a general model behave like a specialist? There is a whole stack of customisation techniques that never touch the weights at all.

The first is RAG. Instead of training documents into the model, the application retrieves them at query time and feeds them into the prompt. The knowledge is current by construction, because it is read from the source on every request rather than frozen at a training date. Adding a document is indexing, not retraining.

The second is context engineering, and the framing is that a good prompt is a carefully assembled bundle of context: the system prompt, the relevant data, format guidelines, worked examples — packaged together deliberately.

That word 'assembled' is doing the work. Context engineering is not writing a better sentence; it is building the input programmatically from parts, each with a job. A system prompt establishing role and constraints. Retrieved evidence. Few-shot examples that pin down format. Explicit output schema. Conversation state, trimmed to what is still relevant. Treated this way the prompt becomes a versioned, testable artefact — which is what makes it improvable rather than a matter of taste.

And it composes with everything else. Every capability you add here is available immediately, works on any model you can switch to, and can be changed by editing a file rather than by running a training job.

KEY POINTS

- RAG retrieves documents at query time instead of training them into weights.
- Retrieved knowledge is current by construction and updated by indexing, not retraining.
- Context engineering assembles the prompt from parts, each with a defined role.
- A programmatically built prompt is versionable and testable rather than a matter of taste.
- These techniques transfer to any model and take effect immediately.

A prompt as an assembled artefact

Every block has a purpose and can be tested independently. Note the ordering: stable content first so it can be cached, variable content last.

```
def build_prompt(question, session):  
    return "\n\n".join([  
        SYSTEM_ROLE,                # who the model is, what it must not do  
        OUTPUT_SCHEMA,              # the exact shape required back  
        FEW_SHOT_EXAMPLES,         # format pinned by demonstration  
        format_evidence(retrieve(question, k=8)), # RAG  
        format_history(session, max_turns=6),    # trimmed state  
        f"Question: {question}",  
    ])  
  
# stable blocks first → cacheable prefix  
# each block independently testable → the prompt is a versioned artefact
```

Knowledge in weights versus knowledge in context

The comparison that explains why most 2023-era fine-tuning has been displaced. Nearly every row favours context for knowledge, which is what fine-tuning was most often used for.

	fine-tuned in	supplied in context
updating a fact	retrain the model	re-index one document
freshness	frozen at training	current on every query
citations	none available	the source is right there
auditing what it knows	not possible	read the retrieved text
removing a document	retrain from scratch	delete from the index
switching base model	redo the whole run	unchanged

Folders of files that package how to do something and which tools to use, loaded on demand when the model meets a matching task.

The third technique in the no-weights stack is agent skills. A skill is a folder of files — commonly markdown — that packages procedural knowledge: how to do something, and the tools to use to do it. The model loads it on demand when it encounters a task that calls for it.

The example makes the point sharply. Instead of fine-tuning a model to know how to write SQL against a very specific schema, a SQL agent skill can tell the model exactly what to do — and any general-purpose model can then use that skill.

This is a genuinely different category from RAG, and the distinction is worth holding. RAG supplies facts: what is true. Skills supply procedure: how to carry out a task in this environment, in what order, with which tools, subject to which conventions. Fine-tuning has historically been used for both, and it is a heavy instrument for either.

The practical properties are what make skills attractive. A skill is a file, so it is readable, diffable and reviewable in a pull request. It is loaded on demand, so a hundred skills cost nothing until one is relevant. It works with any capable model, so it survives a provider change. And it is edited rather than trained, so the loop from noticing a problem to fixing it is minutes rather than weeks — which is what actually determines how good a system becomes over time.

KEY POINTS

- A skill packages procedural knowledge — how to do a task and which tools to use.
- The model loads it on demand when a task calls for it.
- RAG supplies facts; skills supply procedure. Different categories.
- A skill is a reviewable file, not an opaque weight change.
- It transfers across models and is fixed by editing rather than retraining.

A SQL skill instead of a SQL fine-tune

Everything a fine-tune would have encoded implicitly, stated explicitly and readably. When the schema changes, you edit the file — which is a pull request rather than a project.

```
skills/warehouse-sql/
├─ SKILL.md          when to use this, and the procedure to follow
├─ schema.sql        the actual table definitions
└─ conventions.md    house rules: date handling, soft deletes, naming

# SKILL.md, in essence:
# - always filter on `deleted_at IS NULL`
# - dates are stored UTC; report in Europe/Madrid
# - never SELECT *; the fact tables are wide
# - validate with EXPLAIN before running anything over 10M rows

# any capable model can follow this. no training, no artefact to host.
```

Three kinds of missing capability, three different fixes

Diagnose before choosing. Most 'we need to fine-tune' conversations are actually one of the first two rows, and both have much cheaper answers.

what is missing	symptom	fix
facts	makes things up about your domain, or is out of date	RAG
procedure	knows the facts, does the steps in the wrong order or ignores your conventions	agent skill
behaviour at scale	correct but the tone, format or style drifts across thousands of calls	fine-tuning

07 LoRA and parameter-efficient fine-tuning

7:11

Train a small adapter over frozen base weights. Most of what is called fine-tuning in production today is some flavour of this.

Fine-tuning is not free. Beyond the training run itself there is the cost of collecting the examples, evaluating the results, avoiding regressions, and maintaining the custom model as frontier models move on.

LoRA — low-rank adaptation — reduces part of that. A team fine-tunes by training a small adapter that sits on top of an existing base model, while most of the original weights stay locked. A lot of what gets labelled fine-tuning in production today is some flavour of LoRA or a related parameter-efficient method.

The mechanism is worth understanding because it explains the properties. Rather than updating a large weight matrix directly, LoRA learns two much smaller matrices whose product approximates the update, controlled by a rank parameter. Only those small matrices train. The base weights are never modified — the adapter is applied alongside them.

The consequences follow directly, and they change the economics rather than merely the arithmetic. Memory and storage drop enormously, since you save an adapter rather than a model. You can keep several adapters for different tasks against one copy of the base model, and swap them at load time. The base model remains intact underneath, so the adapter can be turned off, which makes the change reversible in a way full fine-tuning is not. And because a run is cheap, you can afford to iterate.

What LoRA does not fix is the part that dominates the cost. You still need the data, the evaluation, the regression checks and the maintenance decision every time the base model moves. LoRA made the training step cheap; it did not make the project cheap.

KEY POINTS

- Fine-tuning's costs are data collection, evaluation, regression avoidance and maintenance — not mainly compute.
- LoRA trains a small adapter while the base weights stay frozen.
- The update is factorised into two low-rank matrices controlled by a rank parameter.
- Adapters are small, swappable, and can be disabled — making the change reversible.
- Most production fine-tuning today is LoRA or a related PEFT method.
- LoRA cheapens the training step, not the data, evaluation or maintenance around it.

What LoRA is actually computing

The frozen weight matrix keeps its original output; the adapter adds a low-rank correction alongside it. B starts at zero, so the adapted model is exactly the base model at step zero and departs from it gradually.

```
# a normal layer:          y = W x          W is d x k, frozen
# with a LoRA adapter:    y = W x + (a/r) B A x
#
#   A : r x k    initialised random    only these train
#   B : d x r    initialised to zero
#   r : the rank – the bottleneck width, and the capacity dial
#   a : alpha – scales the size of the update

# B = 0 at the start, so the adapter contributes nothing until it learns.
# trainable parameters: r(d + k) instead of d x k.
```

Configuring and merging an adapter

Training touches a small fraction of parameters. Merging folds the adapter into the base weights for deployment when you want no runtime overhead — and keeping it unmerged is what lets you serve several specialisations from one base copy.

```
from peft import LoraConfig, get_peft_model

model = get_peft_model(base_model, LoraConfig(
    r=16,
    lora_alpha=32,
    target_modules=["q_proj", "v_proj"],
    lora_dropout=0.05,
    task_type="CAUSAL_LM",
))
model.print_trainable_parameters()

model.save_pretrained("adapters/legal-memo-v3") # small, not a whole model

# serve several specialisations from one base copy:
#   base + adapters/legal-memo-v3
#   base + adapters/support-triage-v1
#   base alone          ← the adapter is removable
```

08 Where fine-tuning still wins

7:52

Three specific bottlenecks the no-weights stack cannot solve: hard latency limits, distillation into smaller models, and training against a programmatic grader.

Fine-tuning is not dead. It survives for a narrower set of reasons than in 2023, and each is a specific bottleneck rather than a general preference.

The first is latency. If a model has to respond in real time — a voice agent answering a phone call — frontier reasoning models, with all their thinking time, are often too slow. When real-time responsiveness is the binding constraint, a small fine-tuned model may still be the right answer. Note what makes this argument work: it is not that the fine-tune is more accurate, it is that a small model is fast enough and fine-tuning is what makes a small model good enough at one narrow task.

The second is distillation. Take a large frontier model, generate high-quality outputs from it, then fine-tune a much smaller model on those outputs — generating reasoning traces from a large teacher and using them to train a smaller student. This is how a small model acquires behaviour it could not learn from raw data, and it inverts the usual data problem: instead of collecting thousands of human-written examples, you generate them.

The third is reinforcement fine-tuning. Instead of training only on fixed correct answers, RFT uses a prompt dataset plus a grader. The model samples candidate answers, the grader scores them, and training updates the model to make high-scoring answers more likely. The catch is that RFT only really works when the output can be programmatically graded — when there is a definitive right answer. Code that must compile and pass tests, a query returning a specific result, a value within tolerance. Where a grader is available this is powerful, because you optimise the thing you actually care about rather than similarity to a reference answer somebody wrote.

KEY POINTS

- Hard latency limits: reasoning models are too slow for real-time interaction, so a small fine-tuned model wins on speed.
- Distillation: a large teacher generates training data, and a small student is fine-tuned on it.
- Distillation inverts the data problem — you generate examples instead of collecting them.
- Reinforcement fine-tuning trains against a grader rather than against fixed answers.
- RFT requires programmatic gradability — a definitive right answer you can check in code.
- Each case is a specific bottleneck, not a general preference for custom weights.

When latency is the binding constraint

A voice agent has a hard budget: past roughly a second of silence the conversation breaks down. A reasoning model that thinks for seven seconds is not slightly worse here, it is unusable — and no amount of accuracy compensates.

```
budget for a voice turn: < ~800 ms end to end
```

frontier reasoning model	2,000–8,000 ms	unusable, however accurate
frontier standard model	600–1,500 ms	marginal, variance is the problem
small fine-tuned model	80–200 ms	fits, with room for TTS

```
# the fine-tune is not more accurate. it makes a fast model
# good enough at one narrow task, which is the whole requirement.
```

Distillation: generate the dataset you cannot collect

Keep only what the teacher got right, verified independently. Distilling unfiltered teacher output trains the student on the teacher's mistakes as faithfully as on its successes.

```
# 1. teacher produces reasoning traces on real inputs
traces = [
    {"input": p, "output": teacher.generate(p, reasoning="extended")}
    for p in production_prompts
]

# 2. keep only verified-correct traces – this filter is essential
clean = [t for t in traces if verify(t["input"], t["output"])]

# 3. fine-tune the small student on what survived
student = finetune(small_base, clean)

# thousands of examples generated rather than collected,
# and the student inherits behaviour, not just facts.
```

Reinforcement fine-tuning against a grader

The grader is the whole design. If you can write it, RFT optimises the property you actually want; if you cannot write it, RFT does not apply — and inventing a fuzzy grader optimises whatever that grader happens to reward.

```
def grade(prompt, answer):
    """Programmatic, definitive. This is the hard part."""
    if not compiles(answer):
        return 0.0
    passed = run_tests(answer)
    return 0.3 + 0.7 * (passed / TOTAL_TESTS)

for prompt in dataset:
    candidates = [model.sample(prompt) for _ in range(k)] # explore
    scores     = [grade(prompt, c) for c in candidates]   # judge
    model.update(prompt, candidates, scores)              # reinforce

# works: code that must compile and pass, queries with a known result,
#        numeric answers within tolerance
# does not work: "write a good summary" — no definitive right answer
```

09 The decision order

10:01

Base model, then prompt and context engineering, then RAG, then skills — and fine-tuning only for a bottleneck the rest cannot clear.

The practical framework runs in a specific order, cheapest and most reversible first.

Start with a base model. Implement prompt and context engineering — often the single largest improvement available, and it costs an afternoon. If the knowledge you need is fresh or proprietary, add RAG. If what is missing is procedural know-how, add agent skills. Only then, if a specific bottleneck remains that none of the stack can solve, reach for fine-tuning.

The order is not arbitrary. Each step is cheaper, faster to evaluate and easier to reverse than the one after it, and each one tells you something you need before taking the next. And critically, the evaluation harness you build while doing prompt engineering is the same instrument that later tells you whether fine-tuning helped — so working in this order means arriving at a training decision already equipped to judge it.

The discipline this imposes is that fine-tuning has to be justified by a named bottleneck. 'The model doesn't know our data' is RAG. 'It ignores our conventions' is a skill. 'It has to answer in under 200 milliseconds' is a bottleneck the rest of the stack genuinely cannot address, and so is 'the output must pass a test suite and we can grade it automatically'. If you cannot name the bottleneck in a sentence, you have not exhausted the stack.

Which leaves fine-tuning in a smaller but real place. Not the first response to a domain problem, as it reasonably was in 2023, but the answer to a specific, articulable constraint — after everything cheaper has been tried and measured.

KEY POINTS

- Order: base model → prompt and context engineering → RAG → agent skills → fine-tuning.
- Each step is cheaper, faster to evaluate and easier to reverse than the next.
- The eval harness built during prompt work is what later judges the fine-tune.
- Fine-tuning requires a named bottleneck stated in one sentence.
- Knowledge gaps are RAG; procedural gaps are skills; latency and gradable outputs are fine-tuning.

The ladder, with what each rung fixes

Work down until the problem goes away. Most teams that jump to the bottom rung discover afterwards that a rung above would have been sufficient — and considerably cheaper.

1. base model	is the general model already good enough? (measure before assuming it is not)
2. prompt & context engineering	role, schema, examples, format, constraints → cost: an afternoon. reversible: instantly.
3. RAG	fresh or proprietary knowledge → cost: an index. reversible: turn it off.
4. agent skills	procedural know-how, conventions, tools → cost: a file. reversible: delete it.
5. fine-tuning	a named bottleneck none of the above clears → cost: weeks. reversible: not really.

The sentence you must be able to write

A blunt test. The valid reasons name a constraint that is measurable and that no prompt, index or file can address. The invalid ones describe a symptom whose cause sits higher up the ladder.

valid

"we must answer in under 200ms and the smallest capable model is not accurate enough on this narrow task without training"

"we can grade outputs programmatically and want to optimise that score directly"

"we need a small self-hosted model to match a large one on one task, and we can generate the training data from the large one"

invalid

"it doesn't know our products" → RAG

"it doesn't follow our SQL conventions" → skill

"the answers aren't good enough" → measure first

"we have a lot of domain data" → not a bottleneck

Glossary

Base model

The off-the-shelf pre-trained model, from a frontier lab or open source, before any customisation.

Fine-tuning

Continuing a base model's training on a focused dataset so the domain behaviour ends up in the weights.

Moving target

The problem that a fixed custom model competes against frontier models that keep improving, so its lead decays without anything changing.

Context window

How much text a model can process in one call. Growth from thousands to millions of tokens removed much of the need to bake knowledge into weights.

Reasoning model

A model that performs extended step-by-step thinking at inference time, shifting capability from training to the moment of the question.

Context engineering

Deliberately assembling the prompt from parts — system role, retrieved data, examples, format rules — as a versioned, testable artefact.

Agent skill

A folder of files packaging procedural knowledge and the tools to apply it, loaded on demand by any capable model.

LoRA

Low-rank adaptation: training two small matrices whose product approximates a weight update, while the base weights stay frozen.

PEFT

Parameter-efficient fine-tuning: the family of methods, including LoRA, that adapt a model by training a small fraction of parameters.

Adapter

The small trained artefact a PEFT method produces. Swappable, stackable against one base model, and removable.

Rank (r)

The bottleneck width of a LoRA adapter, controlling its capacity and its parameter count.

Distillation

Using a large teacher model to generate high-quality outputs, then fine-tuning a smaller student model on them.

Reinforcement fine-tuning (RFT)

Training against a grader rather than fixed answers: the model samples candidates, the grader scores them, and high scores are reinforced.

Grader

A programmatic scoring function used in RFT. Requires a definitive checkable answer, which is what limits where RFT applies.

Check yourself

Answer first, then open each one.

The 2023 blind test showing 97% attorney preference was methodologically sound. Why did its conclusion still expire?

It compared a finished artefact against one snapshot of something under continuous development. The custom model stopped improving the day it shipped while the general model did not, so the measured lead was real but time-bounded — and nobody wrote the time bound down.

Why did growing context windows undermine so much of what fine-tuning was used for in 2023?

Much of that fine-tuning was knowledge injection — putting domain documents into the weights because they would not fit in a 2,000-token prompt. A million-token window does that natively, per query, with nothing to go stale and no training run. The constraint that justified the technique disappeared while the advice stayed in circulation.

What is the difference between what RAG supplies and what an agent skill supplies?

RAG supplies facts — what is true, retrieved at query time. A skill supplies procedure — how to carry out a task in this environment, in what order, with which tools and conventions. A model can have every relevant fact and still do the steps wrong, which is a skill problem and no amount of retrieval fixes it.

Why does a LoRA adapter contribute nothing to the model's output at the start of training?

Because B is initialised to zero, so the product BA is zero and the adapted model is exactly the base model. Training moves it away gradually from that known-good starting point, which is what keeps the adaptation stable rather than perturbing a working model from step one.

Why must distillation filter the teacher's outputs before training the student?

Because the student learns whatever regularity is in the data, and an unfiltered trace set contains the teacher's mistakes alongside its successes. It will reproduce both faithfully. Independent verification of each trace is what makes the generated dataset usable rather than merely large.

Why does reinforcement fine-tuning only work where outputs can be graded programmatically?

The training signal is the grader's score, so a grader that cannot distinguish good from bad optimises the model toward whatever it does reward. Where a definitive check exists — code compiles and passes tests, a query returns the right rows — you optimise the property you actually care about. Where it does not, inventing a fuzzy grader produces a model that games it.

Where to go next

- Build a domain benchmark you own for a task you care about, and score the current frontier models against any custom model you run — this is the only measurement that tells you whether a fine-tune is still earning its cost.
- Take a prompt you use in production and rewrite it as an assembled artefact with separately testable blocks, then measure the change against a fixed set of cases.
- Write an agent skill for a procedure your team performs repeatedly, and compare its results against whatever you do today for the same task.
- Train a LoRA adapter on a few hundred examples, save it, and verify for yourself that it can be loaded and unloaded against the same base model — the reversibility is the property that matters most in practice.
- Pick a task where you can write a definitive grader and estimate what fraction of your use cases meet that bar; the answer tells you how much of the RFT literature is relevant to you.
- Time your last fine-tuning project end to end and compare it against the release cadence of the models you would have been competing with.
- Read the LoRA paper (Hu et al., 2021) for the low-intrinsic-rank argument that explains why small adapters recover most of full fine-tuning's benefit.

Lesson generated from a YouTube transcript with YouTube Lesson Builder.

Explanations are AI-generated. Check anything you intend to rely on.